TECHNICAL DOCUMENTATION · CAPSTONE PROJECT

Capstone RAG Chatbot

A multi-provider Retrieval-Augmented Generation assistant with hybrid conversational memory, built on the **LangChain** framework and the LangChain Expression Language (LCEL).

LangChain 0.3 | LCEL Runnables | OpenAI | Google Gemini | FAISS | Pinecone | Streamlit | Hybrid Memory | Python 3.12



Document	Framework	Programme	Revision
Architecture & Code Reference	LangChain 0.3.x	IIT Patna · Module 5	1.0 · October 2026

Table of Contents

01 Executive Summary

1.1 Purpose · 1.2 Key capabilities · 1.3 At a glance

02 Technology Stack

2.1 Stack by layer · 2.2 LangChain package family

03 Project Structure

3.1 Directory tree · 3.2 Folder reference · 3.3 File reference

04 System Architecture

4.1 Layered view · 4.2 Component responsibilities · 4.3 Design principles · 4.4 Module dependency graph

05 End-to-End Workflow

5.1 Ingestion pipeline · 5.2 Question-answering pipeline · 5.3 Streamlit run lifecycle

06 LangChain Framework Usage

6.1 Usage map · 6.2 Concepts explained · 6.3 Anatomy of the LCEL chain · 6.4 Config propagation

07 Module Reference

7.1 utils · 7.2 loader · 7.3 splitter · 7.4 embedder · 7.5 retriever · 7.6 rag_chain · 7.7 memory · 7.8 streamlit_app · 7.9 template · 7.10 notebook

08 Configuration Reference

8.1 config.json · 8.2 Environment variables

09 Installation & Operation

9.1 Prerequisites · 9.2 Setup · 9.3 Running · 9.4 Switching providers

10 Engineering Review

10.1 Findings register · 10.2 Recommended roadmap

11 Glossary

Key RAG and LangChain terminology

Badge legend: LangChain LangChain component · Core project module · External SDK non-LangChain library · UI presentation layer

> 🦜🔗 **How to read this document**
>
> Every place where the LangChain framework is used is marked with a **green** "LangChain in action" callout, a green-headed table, or a LangChain badge. Code references use the form `modules/rag_chain.py:76` (file : line) so they can be followed directly in the repository.

1 Executive Summary

1.1 Purpose

The **Capstone RAG Chatbot** is a document question-answering assistant. Users upload PDF, DOCX or TXT files (or point the loader at a web URL); the application splits each document into overlapping chunks, converts the chunks into vector embeddings, and stores them in a vector database. When a user asks a question, the most relevant chunks are retrieved and passed — together with a running summary of the conversation and the last few exchanges — to a Large Language Model (LLM), which writes a grounded answer.

The whole pipeline is assembled from **LangChain** building blocks. LangChain supplies the common data model (`Document`), provider-neutral interfaces for embeddings, vector stores, retrievers and chat models, and the **LangChain Expression Language (LCEL)**, which the project uses to compose those parts into one runnable chain with per-session memory.

1.2 Key Capabilities

Capability	Description	Enabled by
Multi-format ingestion	Loads `.txt`, `.md`, `.pdf` (page by page), `.docx` and live web pages into a common document format.	LangChain `Document`, pypdf, python-docx, BeautifulSoup
Smart chunking	Recursive, structure-aware splitting with overlap, with the chunk position kept in each chunk's metadata.	LangChain `RecursiveCharacterTextSplitter`
Pluggable embeddings	Switch between OpenAI and Google Gemini embedding models through one setting in `config.json`.	LangChain `OpenAIEmbeddings`, `GoogleGenerativeAIEmbeddings`
Local or cloud vector store	Saves a FAISS index to disk, or uses a serverless/pod Pinecone index. Detects the embedding dimension and checks it against an existing index.	LangChain `FAISS`, `PineconeVectorStore`
Two retrieval strategies	Plain top-k similarity search, or a *Multi-Query* retriever that rewrites the question into several variants to improve recall.	LangChain `VectorStoreRetriever`, `MultiQueryRetriever`
Swappable LLM per chat	Each chat session can choose OpenAI GPT-4o-mini or Gemini 2.5 Flash at runtime.	LangChain `ChatOpenAI`, `ChatGoogleGenerativeAI`
Hybrid conversational memory	Combines an LLM-written *summary memory* with a rolling window of the last *n* Q&A turns, kept separately for each session.	LangChain `ConversationSummaryMemory`, `RunnableWithMessageHistory`
Multi-session chat UI	Streamlit interface with several independent chats, a provider choice and a memory toggle for each chat, and document upload.	UI Streamlit

1.3 At a Glance

7	~750	16	7	2	2
CORE MODULES	LINES OF PYTHON	LANGCHAIN CLASSES USED	LANGCHAIN PACKAGES	LLM PROVIDERS	VECTOR STORES

2 Technology Stack

2.1 Stack by Layer

Layer	Technology	Package (pinned)	Role in the project
Orchestration	LangChain LangChain framework	`langchain==0.3.25`	Legacy entry points: `MultiQueryRetriever`, `ConversationSummaryMemory`, prompt and runnable re-exports.
	LangChain LangChain Core	`langchain-core==0.3.65`	Base abstractions: `Document`, Runnables (LCEL), prompts, chat history, `RunnableWithMessageHistory`.
	LangChain Text Splitters	`langchain-text-splitters==0.3.8`	`RecursiveCharacterTextSplitter`.
	LangChain Community	`langchain-community==0.3.24`	`FAISS` (and `Chroma`) vector-store wrappers.
LLM providers	LangChain OpenAI integration	`langchain-openai==0.3.18`	`ChatOpenAI` chat model, `OpenAIEmbeddings`.
	LangChain Google GenAI integration	`langchain-google-genai≥2.1.5`	`ChatGoogleGenerativeAI`, `GoogleGenerativeAIEmbeddings`.
Vector DB	LangChain Pinecone integration	`langchain-pinecone==0.2.13`	`PineconeVectorStore` wrapper; also brings in the `pinecone` SDK.
	External SDK FAISS (CPU)	`faiss-cpu==1.8.0.post1`	In-process similarity search engine behind the LangChain `FAISS` wrapper.
	External SDK Pinecone SDK	`pinecone` (unpinned)	Index admin: `list_indexes`, `create_index`, `describe_index`.
Document parsing	External SDK pypdf · python-docx · BeautifulSoup · requests	`pypdf==5.1.0`, `python-docx==1.1.2`, `beautifulsoup4==4.12.3`, `requests==2.32.3`	Text extraction from PDF, Word and HTML pages.
Presentation	UI Streamlit (Gradio placeholder)	`streamlit==1.39.0`, `gradio==5.9.1`	Chat interface, sidebar session management, document upload.
Configuration	python-dotenv · JSON	`python-dotenv==1.0.1`	Loads API keys from `config/.env`; reads runtime settings from `config/config.json`.
Observability / utils	LangSmith · tiktoken · numpy · pandas	`langsmith==0.3.45`, `tiktoken==0.7.0`	Tracing (when `LANGCHAIN_TRACING_V2` is set), tokenisation, numerical support.

2.2 The LangChain Package Family — Why Several Packages?

Since version 0.2, LangChain has been split into separate packages. The project uses this split as intended: provider-specific code lives in *partner packages*, while the project's own logic depends only on the stable interfaces in `langchain-core`.

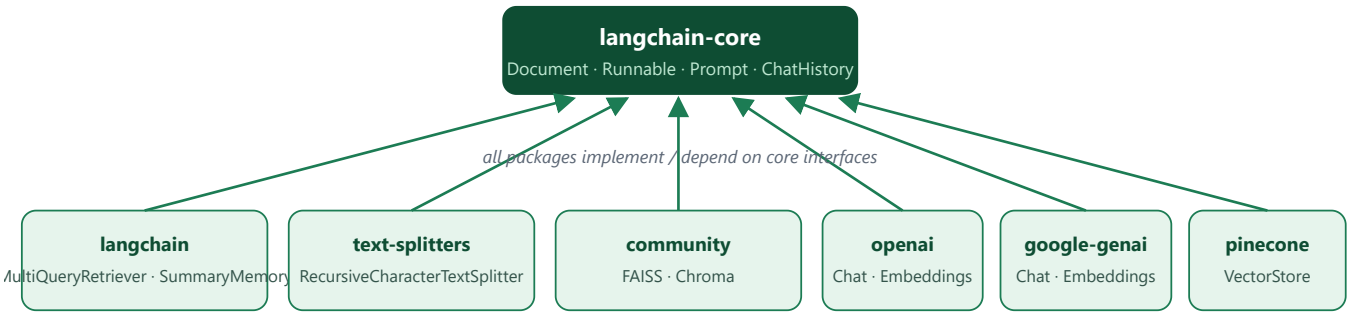


Figure 2-1 · The LangChain packages used by the project and how they depend on `langchain-core`.

3 Project Structure

3.1 Directory Tree

```
Capstone-Project-RAG-Application/
├── app/                              # Presentation layer (user interfaces)
│   ├── streamlit_app.py              # ★ Main multi-session chat UI + pipeline orchestration
│   └── gradio_app.py                 # Placeholder for an alternative Gradio UI (empty)
├── config/                           # Runtime configuration
│   ├── config.json                   # Provider, vector-store, retriever, chunking settings
│   └── .env                          # API keys (git-ignored; created from env-sample.txt)
├── data/                             # Source documents
│   ├── sample_docs/                  # Reference corpus used by the notebook
│   │   ├── sample_text.txt           #   scikit-learn article (~6 KB)
│   │   └── sample_text2.txt          #   LLM / LangChain article (~15 KB)
│   └── uploaded_docs/                # Files saved by the Streamlit uploader
├── modules/                          # Core RAG library (business logic)
│   ├── __init__.py                   # Marks the folder as a Python package
│   ├── utils.py                      # 🐔 .env loading + embedding-model factory
│   ├── loader.py                     # 🐔 File / URL → LangChain Document
│   ├── splitter.py                   # 🐔 Document → chunks (RecursiveCharacterTextSplitter)
│   ├── embedder.py                   # 🐔 Chunks → FAISS / Pinecone vector store
│   ├── retriever.py                  # 🐔 Vector store → Base / Multi-Query retriever
│   ├── rag_chain.py                  # 🐔 LCEL RAG chain (prompt + LLM + context + memory)
│   └── memory.py                     # 🐔 Hybrid summary + rolling-window session memory
├── notebooks/
│   ├── RAG_Debugging_Notebook.ipynb  # Step-by-step test harness for every module
│   └── vectorstores/openai_faiss_index/ # FAISS index created when running the notebook
│       ├── index.faiss               #   binary vector index
│       └── index.pkl                 #   pickled docstore + id mapping
├── vectorstores/                     # (runtime) FAISS index created by the app — cwd-relative
├── .vscode/settings.json             # Editor settings
├── env-sample.txt                    # Template for config/.env
├── requirements.txt                  # Pinned Python dependencies
├── run_app.bat                       # Windows launcher (placeholder, empty)
├── template.py                       # Scaffolding script that generated this layout
└── README.md                         # Repository landing page
```

3.2 Folder Reference

Folder	Layer	Purpose	Key contents
`app/`	UI	Entry points that users interact with. Holds the Streamlit application, which also wires the whole pipeline together.	`streamlit_app.py`, `gradio_app.py`
`modules/`	Core	Reusable RAG library. Each file covers one pipeline stage (load → split → embed → retrieve → generate → remember). Contains no UI code, so it can be reused from notebooks, scripts or other front ends.	7 modules + `__init__.py`
`config/`	Config	Keeps configuration separate from code: non-secret settings in JSON, secrets in `.env`.	`config.json`, `.env`

Folder	Layer	Purpose	Key contents
`data/`	Data	Raw input documents. `sample_docs` holds a fixed test corpus; `uploaded_docs` stores every file uploaded through the UI.	`.txt` samples
`notebooks/`	Dev / QA	Interactive debugging and testing of each module on its own, plus a FAISS index produced while running the notebook.	`RAG_Debugging_Notebook.ipynb`
`vectorstores/`	Data	Saved FAISS indexes. The path is `<cwd>/vectorstores/<provider>_<type>_index`, so where it ends up depends on the directory the process is started from.	`index.faiss`, `index.pkl`
`.vscode/`	Tooling	Workspace editor preferences.	`settings.json`

3.3 File Reference

File	LOC	Purpose	LangChain usage
`modules/utils.py`	30	Loads `config/.env` when imported; factory that returns the configured embedding model.	LC `OpenAIEmbeddings`, `GoogleGenerativeAIEmbeddings`
`modules/loader.py`	62	Reads TXT/MD/PDF/DOCX files and URLs and converts them to `Document` objects with a `source` metadata field.	LC `langchain_core.documents.Document`
`modules/splitter.py`	16	Splits documents into overlapping chunks and records `chunk_index` in each chunk's metadata.	LC `RecursiveCharacterTextSplitter`, `Document`
`modules/embedder.py`	156	Creates or updates a FAISS index on disk, or creates/connects to a Pinecone index (with dimension checking) and upserts the chunks.	LC `FAISS`, `PineconeVectorStore`, `Document`
`modules/retriever.py`	31	Wraps the vector store in a retriever — either plain similarity search or LLM-powered multi-query.	LC `as_retriever`, `MultiQueryRetriever`, chat models
`modules/rag_chain.py`	95	Builds the LCEL chain: parallel context + memory gathering → prompt → chat model.	LC `ChatPromptTemplate`, `RunnableParallel`, `RunnableLambda`, `ChatOpenAI`, `ChatGoogleGenerativeAI`
`modules/memory.py`	112	Per-session message history, summary memory and a rolling window of recent turns; wraps the chain so memory is updated after each turn.	LC `InMemoryChatMessageHistory`, `ConversationSummaryMemory`, `RunnableLambda`, `RunnableWithMessageHistory`
`modules/__init__.py`	0	Package marker so modules can be imported as `modules.xxx`.	—

File	LOC	Purpose	LangChain usage
`app/streamlit_app.py`	181	UI + orchestration: session management, document upload and ingestion, building the chain, chat loop.	Uses the LCEL chain via `.invoke(.., config={"configurable": {...}})`
`app/gradio_app.py`	0	Reserved for an alternative Gradio front end (not implemented).	—
`config/config.json`	35	Runtime settings (providers, vector store, retriever, chunking).	Selects which LangChain integrations are created
`env-sample.txt`	5	Template listing the API keys needed in `config/.env` .	—
`requirements.txt`	44	Pinned dependency list for a reproducible environment.	Declares 7 LangChain packages
`template.py`	68	Scaffolding script that creates the folders and empty files.	—
`notebooks/RAG_Debugging_Notebook.ipynb`	10 cells	Runs load → split → embed → search → retrieve → chain + memory one step at a time.	Calls every LangChain-backed module
`run_app.bat`	0	Reserved Windows launcher (empty).	—
`README.md`	—	Repository landing page linking to this documentation.	—

4 System Architecture

4.1 Layered Architecture

The system has four layers. The **presentation layer** handles user interaction. The **orchestration layer** (the body of `streamlit_app.py`) calls the pipeline stages in order. The **core RAG library** (`modules/`) contains one module per stage. The **provider layer** holds the external AI services and vector databases, reached only through LangChain's provider-neutral interfaces.

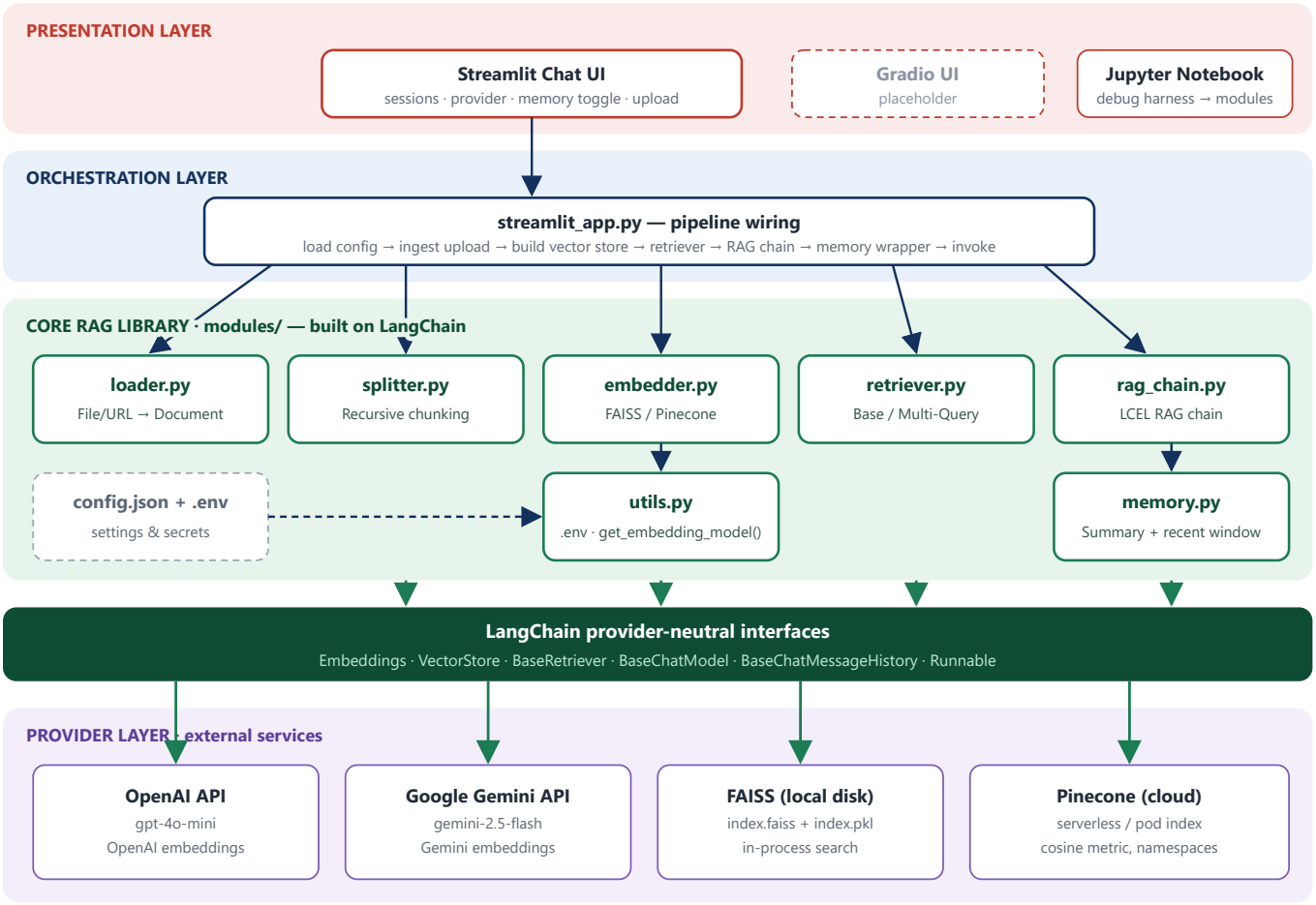


Figure 4-1 · Layered architecture. Arrows show call direction. The core library reaches every external service only through LangChain interfaces (dark-green band).

4.2 Component Responsibilities

Component	Input	Output	Responsibility
Loader	File path or URL (`str`)	`List[Document]`	Turns any supported source into plain text plus `source` metadata. PDFs produce one Document per page.
Splitter	`List[Document]`	`List[Document]` (chunks)	Produces chunks small enough to embed and retrieve, with overlap so meaning is not cut off at chunk edges.

Component	Input	Output	Responsibility
Embedder	Chunks + config	`VectorStore`	Embeds chunks and saves them to FAISS or Pinecone; loads an existing index and adds new chunks to it.
Retriever	`VectorStore` + config	`BaseRetriever`	Returns the top-k most relevant chunks for a query (optionally widened by multi-query rewriting).
RAG Chain	Retriever + config	LCEL `Runnable`	Gathers context and memory in parallel, fills the prompt, calls the chat model.
Memory	RAG chain + session id	`RunnableWithMessageHistory`	Records every turn, keeps the summary up to date, maintains the recent-turn window, and keeps sessions separate.
Utils	Provider name	`Embeddings`	Loads secrets and returns the matching embedding model.
UI / Orchestrator	User actions	Rendered chat	Manages chat sessions and per-session settings, ingests uploads, calls the chain.

4.3 Design Principles

PROVIDER-NEUTRAL INTERFACES

Every external service is accessed through a LangChain base class (`Embeddings` , `VectorStore` , `BaseChatModel` , `BaseRetriever`). Switching from OpenAI to Gemini, or from FAISS to Pinecone, means changing a config value — the downstream code stays the same.

CONFIGURATION-DRIVEN

`config.json` decides which embedding provider, vector store, retriever and LLM are used; `.env` holds the secrets. Factory functions (`get_embedding_model` , `build_or_update_vectorstore` , `get_retriever` , `build_rag_chain`) read the config and build the matching objects.

COMPOSITION WITH LCEL

The chain is built with the `|` operator and a `RunnableParallel` map rather than as one large function. Every stage is a `Runnable` , so it supports `invoke` , `batch` and `stream` , and is traced automatically in LangSmith.

ONE MODULE PER PIPELINE STAGE

Each pipeline stage is a separate module with a small functional API. The notebook can test each stage independently, and a new front end (for example Gradio) can reuse the whole library without changes.

SESSION ISOLATION

All memory stores are dictionaries keyed by a session UUID, and the session id is passed through LangChain's `configurable` runtime config, so several chats can run side by side without sharing context.

INCREMENTAL INDEXING

An existing FAISS index is loaded and new chunks are added to it (`add_documents`) instead of rebuilding it. For Pinecone, the index is created once and later uploads are upserted.

4.4 Module Dependency Graph

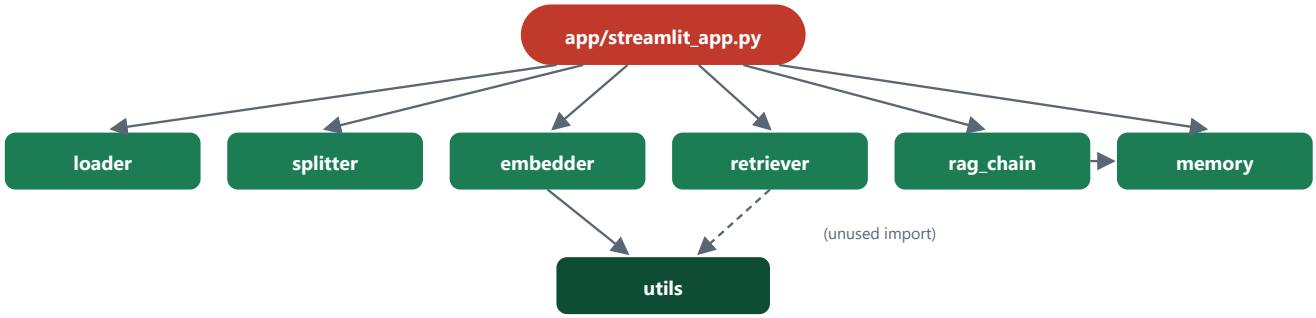
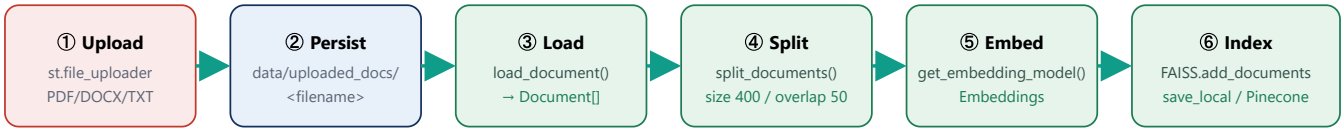


Figure 4-2 · Import graph. `rag_chain` imports `memory` to read summary and recent context; `retriever` imports `utils` but does not use it.

5 End-to-End Workflow

The application has two pipelines: an **ingestion pipeline** that runs when a document is uploaded, and a **question-answering pipeline** that runs for every chat message. Both run inside Streamlit's top-to-bottom script execution model (section 5.3).

5.1 Ingestion Pipeline (Indexing)



Steps ③–⑥ use LangChain abstractions (green) · triggered in app/streamlit_app.py lines 111–124

Figure 5-1 · Ingestion pipeline.

#	Stage	Code	What happens
1	Upload	`streamlit_app.py:102`	User selects a file; Streamlit holds it in memory as an `UploadedFile`.
2	Persist	`streamlit_app.py:113–118`	File bytes are written to `data/uploaded_docs/<name>`, creating the folder if needed.
3	Load LC	`loader.load_document`	Chooses a parser by extension or URL scheme and returns `Document(page_content, metadata={"source": ...})` objects.
4	Split LC	`splitter.split_documents`	`RecursiveCharacterTextSplitter(400, 50)` splits each document and adds `chunk_index` to the metadata.
5	Embed LC	`utils.get_embedding_model`	Creates `OpenAIEmbeddings` or `GoogleGenerativeAIEmbeddings` based on `embedding_provider`.
6	Index LC	`embedder.build_or_update_vectorstore`	FAISS: load the existing index and call `add_documents`, or `from_documents` for a new index → `save_local`. Pinecone: create/verify the index → `PineconeVectorStore.from_documents` (upsert).

5.2 Question-Answering Pipeline (Retrieval + Generation)

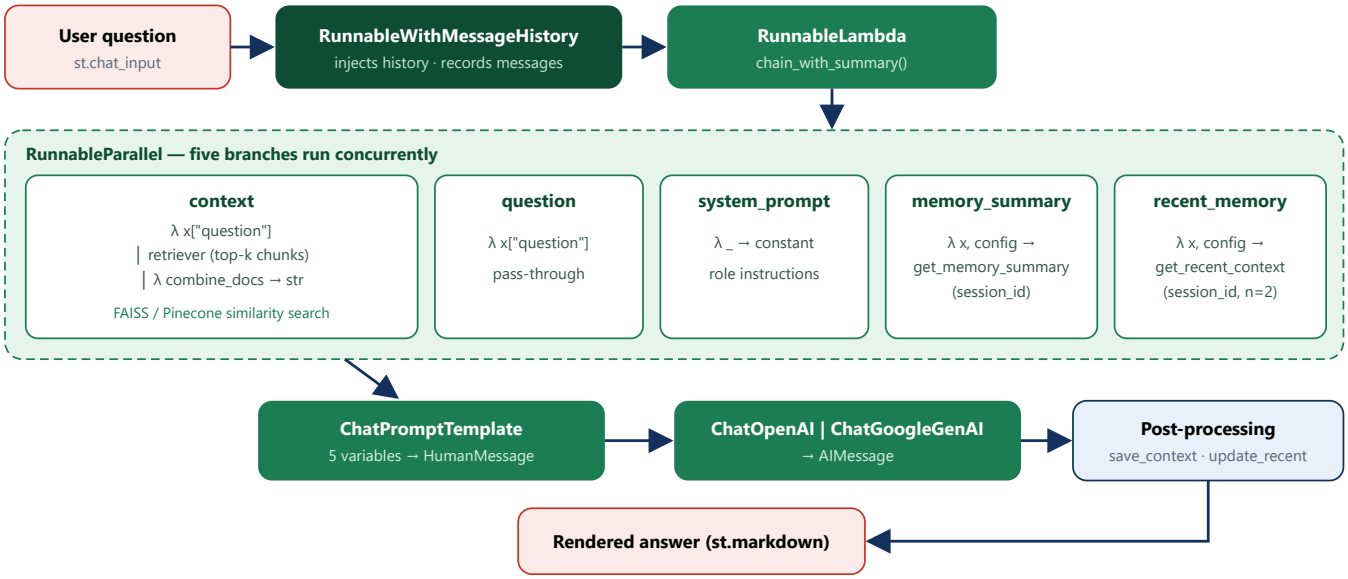


Figure 5-2 · Question-answering pipeline. Every dark-green box is a LangChain Runnable.

#	Step	Detail
1	Invoke with session	`rag_chain_with_memory.invoke({"question": prompt}, config={"configurable": {"session_id": sid}})` — `streamlit_app.py:168`.
2	History injection	`RunnableWithMessageHistory` loads the session's `InMemoryChatMessageHistory`, adds it to the input as `history`, and records the human message.
3	Wrapper lambda	`chain_with_summary(input_data, config)` calls the inner RAG chain and passes the *same* config on, so the session id reaches the inner branches.
4	Parallel fan-out	`RunnableParallel` runs the five branches concurrently in a thread pool: retrieval, question pass-through, system prompt, summary memory and recent-turn memory.
5	Retrieval	The retriever embeds the question with the same embedding model and returns the k nearest chunks; `combine_docs` joins them with blank lines (or returns *"No relevant context found."*).
6	Prompt formatting	`ChatPromptTemplate.from_template` fills the five variables into a single human message.
7	Generation	The chat model chosen for the session produces an `AIMessage`.
8	Memory update	The wrapper saves the turn to summary memory (which asks the LLM to update the running summary), adds it to the rolling window, and returns the response. `RunnableWithMessageHistory` then records the AI message.
9	Render	The UI reads `response.content`, displays it, and adds it to the session's visible chat history.

5.3 Streamlit Run Lifecycle

Streamlit runs `streamlit_app.py` from top to bottom on **every** user interaction (button click, toggle, chat message). Knowing this order is important for understanding how the application behaves and performs:

Order	Phase	Lines	Behaviour on each rerun
1	Bootstrap	1–23	Adds the project root to `sys.path`, imports modules (the first import of `utils` loads `.env`), sets the page config, reads `config.json`.
2	Session state init	27–57	Creates `chat_sessions` / `active_session` once; the *New Chat* button adds a UUID-keyed session.
3	Sidebar controls	60–104	Session selector, per-session LLM provider, per-session memory toggle, file uploader.
4	Ingestion	111–124	Runs whenever the uploader holds a file (see finding F-01 in section 10).
5	Backend build	129–144	Loads the vector store (`docs=[]`), builds the retriever, applies the session's provider to `CONFIG`, builds the RAG chain and memory wrapper. Any exception stops the page with an error.
6	Chat render	150–182	Shows earlier messages; on new input, calls the chain and adds both messages to the session's history.

6 LangChain Framework Usage

This chapter lists every place the project uses LangChain, explains each LangChain concept in the context of this codebase, and then walks through the central LCEL chain in detail.

6.1 LangChain Usage Map

#	LangChain class / API	Import path	Used in	What it does in this project
1	`Document`	`langchain_core.documents`	loader:10 · splitter:16 · embedder	The standard data unit: `page_content` + `metadata`. Every stage takes and returns Documents.
2	`RecursiveCharacterTextSplitter`	`langchain_text_splitters` / `langchain.text_splitter`	splitter:8	Splits text hierarchically on `\n\n`, `\n`, space, then characters, until each chunk is within `chunk_size`.
3	`OpenAIEmbeddings`	`langchain_openai`	utils:23	Converts text to vectors with OpenAI's embedding API (default model, 1536-dim).
4	`GoogleGenerativeAIEmbeddings`	`langchain_google_genai`	utils:29	Converts text to vectors with a Gemini embedding model.
5	`FAISS`	`langchain_community.vectorstores`	embedder:64-79	`from_documents`, `load_local`, `add_documents`, `save_local`, `similarity_search`.
6	`Chroma`	`langchain_community.vectorstores`	embedder:13	Imported for future local-store support; not used yet.
7	`PineconeVectorStore`	`langchain_pinecone`	embedder:148	`from_documents` embeds and upserts chunks into a Pinecone index/namespace.
8	`VectorStore.as_retriever()`	`langchain_core.vectorstores`	retriever:15,28	Wraps a store in a `VectorStoreRetriever` (a Runnable) with `search_kwargs={"k": 3}`.
9	`MultiQueryRetriever`	`langchain.retrievers`	retriever:27	Uses an LLM to write alternative phrasings of the question, retrieves for each, and returns the de-duplicated union.
10	`ChatOpenAI`	`langchain_openai`	rag_chain:22 · retriever:21 · memory:30	Chat model for answers (gpt-4o-mini, streaming), query rewriting, and summarisation.
11	`ChatGoogleGenerativeAI`	`langchain_google_genai`	rag_chain:28 · retriever:23	Gemini 2.5 Flash chat model, used when `llm_provider="gemini"`.
12	`ChatPromptTemplate`	`langchain.prompts` (→ core)	rag_chain:46	Template with five variables, turned into chat messages.

#	LangChain class / API	Import path	Used in	What it does in this project
13	`RunnableParallel`	`langchain.schema.runnable` (→ core)	rag_chain:77	Runs the five input branches concurrently and returns a dict.
14	`RunnableLambda`	`langchain.schema.runnable` (→ core)	rag_chain:79-87 · memory:94	Turns plain Python functions into Runnables; functions that take a `config` parameter receive the runtime config.
15	`InMemoryChatMessageHistory`	`langchain_core.chat_history`	memory:23	In-process list of `HumanMessage` / `AIMessage` objects for each session.
16	`ConversationSummaryMemory`	`langchain.memory`	memory:29	Keeps an LLM-generated running summary of the conversation.
17	`RunnableWithMessageHistory`	`langchain_core.runnables.history`	memory:96	Wraps a Runnable so each call reads and writes a session-scoped chat history.
18	`PromptTemplate`, `LLMChain`	`langchain.prompts`, `langchain.chains`	retriever:2-3	Imported but unused (left over from an earlier version).

6.2 LangChain Concepts Explained in Project Context

6.2.1 Document — the common data unit

A `Document` is a small object with two fields: `page_content: str` and `metadata: dict`. All loaders in `loader.py` produce Documents, the splitter turns one Document into many, and the vector store keeps the metadata next to each vector so retrieved chunks can be traced back to their source file and page.

```python
# loader.py:10 — the factory every loader goes through
def text_to_doc(text: str, source: str = "inline") -> Document:
    return Document(page_content = text, metadata = {"source": source})
```

6.2.2 Text Splitters — chunking for retrieval

Embedding models have input limits, and retrieval works better when each chunk covers one idea. `RecursiveCharacterTextSplitter` tries separators in order `["\n\n", "\n", " ", ""]`: it first splits on paragraphs, splits further only where a piece is still larger than `chunk_size`, then merges small pieces back up to the limit while keeping `chunk_overlap` characters shared between neighbouring chunks.

6.2.3 Embeddings — the provider-neutral vector interface

Both `OpenAIEmbeddings` and `GoogleGenerativeAIEmbeddings` implement the `Embeddings` interface, which has two methods: `embed_documents(list[str])` for indexing and `embed_query(str)` for searching. `embedder.py` uses this interface to measure the vector size (`len(embed_documents(["Hello world!"])[0])`) before creating a Pinecone index.

6.2.4 Vector Stores — FAISS and Pinecone behind one interface

Operation	FAISS (local)	Pinecone (cloud)
Create from chunks	`FAISS.from_documents(docs, emb)`	`PineconeVectorStore.from_documents(docs, embedding=emb, index_name, namespace)`
Load existing	`FAISS.load_local(dir, embeddings, allow_dangerous_deserialization=True)`	Index already exists on the server — `from_documents` connects to it
Add new chunks	`vectorstore.add_documents(docs)`	Upsert happens inside `from_documents`
Persist	`save_local(dir)` → `index.faiss` + `index.pkl`	Automatic (managed service)
Search	Same API for both: `similarity_search(query, k)` / `as_retriever()`	

> ⚠ **Security note — `allow_dangerous_deserialization=True`**
>
> FAISS saves its docstore with Python `pickle`. LangChain requires this flag as an explicit acknowledgement, because loading a pickle file from an untrusted source can execute arbitrary code. That is acceptable here because the index is always created locally by the application. Never load an `index.pkl` received from someone else.

6.2.5 Retrievers — from vectors to relevant chunks

A retriever is any Runnable that takes a query string and returns `List[Document]`. Because it is a Runnable, it can be placed directly in an LCEL pipe (`... | retriever | ...`), which is how `rag_chain.py` uses it. The project supports two kinds:

BASE RETRIEVER

`vectorstore.as_retriever(search_kwargs={"k":3})`. Embeds the question once and returns the 3 nearest chunks. One embedding call, no LLM call — fast and inexpensive.

MULTIQUERYRETRIEVER

An LLM (temperature 0) writes several alternative phrasings of the question; each is searched separately and the results are merged and de-duplicated. Improves recall for vague or ambiguous questions, at the cost of one extra LLM call and several searches.

6.2.6 Chat Models — interchangeable LLMs

`ChatOpenAI` and `ChatGoogleGenerativeAI` both implement `BaseChatModel`: they take a list of messages (or a `PromptValue`) and return an `AIMessage`. `build_rag_chain` chooses one based on `llm_provider` and places it at the end of the pipe; the rest of the chain is identical for both. `streaming=True` is set so a later UI change can use `chain.stream()` to show tokens as they arrive.

6.2.7 Prompt Templates

`ChatPromptTemplate.from_template( ... )` creates a template containing a single human message with the placeholders `{system_prompt}`, `{memory_summary}`, `{recent_memory}`, `{context}` and `{question}`. It is a Runnable that takes a dict with those keys — exactly what the preceding `RunnableParallel` returns.

6.2.8 LCEL Runnables — the composition layer

Primitive	Semantics	Where used
`a | b` (RunnableSequence)	Passes the output of `a` as the input of `b`. The combined object is itself a Runnable.	context branch; `parallel | prompt | llm`
`RunnableParallel({...})`	Sends the same input to every branch, runs them concurrently, and returns a dict of their outputs.	`rag_chain.py:77`
`RunnableLambda(fn)`	Wraps a Python function. If `fn` has a parameter named `config`, LangChain passes in the current `RunnableConfig`.	question/context extraction, memory lookups, `chain_with_summary`
`RunnableWithMessageHistory`	Adds stored history to the input, runs the wrapped Runnable, and saves the new messages, keyed by `configurable.session_id`.	`memory.py:96`

6.2.9 Memory — chat history and summary memory

`InMemoryChatMessageHistory` is the low-level store (a list of messages). `ConversationSummaryMemory` is a higher-level helper from the classic `langchain.memory` API. It shares the same message history and uses an LLM to keep a short running summary up to date (`save_context` → `predict_new_summary`). The project adds its own *rolling window* of the last *n* turns (plain Python), giving **hybrid memory**: the summary keeps long-range context compact, and the window keeps the exact wording of the latest exchanges.

6.3 Anatomy of the LCEL RAG Chain

`modules/rag_chain.py · lines 76–92`

```
rag_chain = (
    RunnableParallel(
        {
            "context":  RunnableLambda(lambda x: x["question"]) | retriever |
RunnableLambda(combine_docs),
            "question": RunnableLambda(lambda x: x["question"]),
            "system_prompt": RunnableLambda(lambda _: system_prompt),
            "memory_summary": RunnableLambda(
                lambda x, config: get_memory_summary(config["configurable"]["session_id"])
            ),
            "recent_memory": RunnableLambda(
                lambda x, config: get_recent_context(config["configurable"]["session_id"])
            ),
        }
    )
    | prompt     # ChatPromptTemplate  : dict  → ChatPromptValue
    | llm        # ChatOpenAI / Gemini : prompt → AIMessage
)
```

Stage	Input type	Output type	Notes
Chain input	—	`{"question": str, "history": [...]}`	`history` is added by `RunnableWithMessageHistory`; the prompt does not use it.
RunnableParallel	input dict	`{context, question, system_prompt, memory_summary, recent_memory}`	Branches run concurrently in a thread pool.
prompt	5-key dict	`ChatPromptValue` (1 HumanMessage)	Placeholders filled with plain string formatting.
llm	`ChatPromptValue`	`AIMessage`	`.content` holds the answer text.

6.4 How the Session ID Reaches the Memory Lookups

The session id is never part of the chain's *input data*; it travels in LangChain's **runtime config**, which LCEL automatically passes down to every nested Runnable:

```
# 1. UI passes the id in config (streamlit_app.py:168)
rag_chain_with_memory.invoke({"question": prompt},
    config={"configurable": {"session_id": st.session_state.active_session}})

# 2. RunnableWithMessageHistory reads configurable.session_id to pick the history
# 3. It calls RunnableLambda(chain_with_summary) — which takes `config`, so LangChain passes it in
def chain_with_summary(input_data, config=None):
    response = rag_chain.invoke(input_data, config=config)   # 4. forwarded explicitly

# 5. Inside RunnableParallel, these two-parameter lambdas receive the same config
RunnableLambda(lambda x, config: get_memory_summary(config["configurable"]["session_id"]))
```

> 🦜🔗 **LangChain in action — why config, not input?**
>
> Keeping the session id in `configurable` leaves the chain's input contract simple (`{"question": ...}`). Callbacks and tracing tags travel the same way, so in LangSmith every step of a session's run appears under one trace with the session metadata attached.

7 Module Reference

Each module below is documented with its purpose, imports, and a reference card for every function (signature, parameters, return value, behaviour), followed by notes on how it uses LangChain.

7.1 `modules/utils.py` — Environment & Embedding Factory

Purpose. Loads secrets and creates the embedding model. When the module is *first imported* it resolves `<project>/config/.env` relative to its own location (so it works from any working directory), loads it with `python-dotenv`, and prints the path it loaded.

Imports	`os`, `dotenv.load_dotenv`, LC `langchain_openai.OpenAIEmbeddings`, LC `langchain_google_genai.GoogleGenerativeAIEmbeddings`
Side effects at import	Loads `.env` into the process environment; prints `Loaded .env from: …`
Used by	`embedder.py` (and imported, unused, by `retriever.py`)

`get_embedding_model(provider: str) → Embeddings` · line 15

provider	Env var required	Returns
`"openai"`	`OPENAI_API_KEY`	`OpenAIEmbeddings(api_key=…)` — library default model
`"gemini"` / `"google"`	`GOOGLE_API_KEY`	`GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-2", google_api_key=…)`
anything else	—	raises `ValueError("Unsupported embedding provider")`

The provider name is lower-cased first. A missing key raises `ValueError` before any network call, so configuration mistakes fail early with a clear message.

LangChain in action

This is a **factory over the `Embeddings` interface**. Callers receive an object with `embed_documents` / `embed_query` and do not need to know which vendor is behind it — this is what lets `embedder.py` handle both providers with the same code.

7.2 `modules/loader.py` — Document Loading

Purpose. Converts files and web pages into LangChain `Document` objects. The project uses small custom loaders built on pypdf, python-docx and BeautifulSoup rather than LangChain's `document_loaders`, but still outputs the standard `Document` type so the rest of the pipeline works unchanged.

Function	Returns	Behaviour
`text_to_doc(text, source="inline")` line 10	`Document`	Builds a LC `Document` with `metadata={"source": source}`. Every loader uses this.
`load_txt(path)` line 13	`[Document]`	Raises `FileNotFoundError` if missing; reads UTF-8 with `errors="ignore"` (undecodable bytes are dropped); source = absolute path.
`load_pdf(path)` line 20	`[Document] × pages`	`pypdf.PdfReader`; **one Document per page**; empty text for image-only pages; source = `"<abs path>: page: N"` (1-based).
`load_docx(path)` line 28	`[Document]`	`python-docx`; keeps non-blank paragraphs joined by `\n\n` (tables, headers and footers are not extracted).
`load_url(url)` line 34	`[Document]`	`requests.get` (15 s timeout, raises on HTTP errors) → BeautifulSoup; extracts text from `p, h1-h3, li` elements; source = URL.
`load_document(path_or_url)` line 45	`[Document]`	**Dispatcher.** `http(s)://` → URL; otherwise checks the file exists and routes by extension: `.txt` → txt, `.pdf` → pdf, `.docx/.docs/.doc` → docx, anything else → tries plain text, otherwise `ValueError`.

> **i Detail**
>
> The extension list at line 53 contains `"md"` without a leading dot, so `.md` files never match that branch. They still load correctly through the plain-text fallback. Legacy binary `.doc` files are not supported by python-docx and will raise an error.

7.3 `modules/splitter.py` — Chunking

```
split_documents(docs: List[Document], chunk_size: int = 500, chunk_overlap: int = 100) → List[Document]  ·  line 7
```

1. Creates a LC `RecursiveCharacterTextSplitter(chunk_size, chunk_overlap)`.
2. For each input document, calls `splitter.split_text(doc.page_content)`.
3. For each chunk, copies the parent metadata and adds `chunk_index` (0-based position within that document).
4. Returns a flat list of new `Document` chunks.

Caller	chunk_size	chunk_overlap
Function default	500	100
Streamlit app (`streamlit_app.py:121`)	400	50
Notebook	200	20 / 40

🦜🔗 **LangChain in action**

Using `split_text` and rebuilding Documents by hand (instead of `split_documents`) gives full control over metadata — every chunk carries `source` + `chunk_index`, so a retrieved chunk can be traced to an exact position in its source. Note: the class is imported twice (lines 3–4); the second import from the older `langchain.text_splitter` path overrides the first. Both point to the same class.

7.4 `modules/embedder.py` — Vector Store Management

Purpose. The single entry point for creating, loading, updating and saving the vector index for whichever backend is configured.

```
build_or_update_vectorstore(docs: List[Document], config: dict, persist_dir: Optional[str] = None) → VectorStore  ·  line 23
```

Parameter	Meaning
`docs`	Chunks to add. Pass `[]` to only load the existing index (this is what the app does on every rerun).
`config`	Parsed `config.json`; reads `embedding_provider` and `vector_store.type` / `vector_store.cloud.*`.
`persist_dir`	FAISS folder; default `<cwd>/vectorstores/<provider>_<type>_index`.

FAISS branch (lines 53–84)

State	Action
`index.faiss` exists	LC `FAISS.load_local(..., allow_dangerous_deserialization=True)`; if `docs` is non-empty → `add_documents(docs)`.
No index, `docs` empty	Raises `ValueError("No documents provided and FAISS index not found.")` — the UI shows this until the first upload.
No index, `docs` provided	LC `FAISS.from_documents(docs, embedding_model)`.
Always	`save_local(persist_dir)`, then returns the store.

Pinecone branch (lines 89–157)

1. Reads `PINECONE_API_KEY` (required) and the region from `PINECONE_ENV`, falling back to `vector_store.cloud.environment`.
2. Index name = `<cloud.index_name>-<provider>` (e.g. `rag-assistant-index-openai`), so each embedding provider gets its own index.
3. **Detects the embedding dimension** by embedding `"Hello world!"` (falls back to `embed_query`).
4. If the index does not exist: creates it with `metric="cosine"` — a `PodSpec` for `*-free` / `*-starter` environments, otherwise `ServerlessSpec(cloud="aws", region=env)`.
5. If it exists: `describe_index` and **checks that the dimension matches**; a mismatch raises a descriptive `ValueError`.
6. LC `PineconeVectorStore.from_documents(docs, embedding, index_name, namespace, pinecone_api_key)` — connects and upserts.

🦜🔗 **LangChain in action**

- **Two backends, one return type:** both branches return a LangChain `VectorStore`, so `retriever.py` calls `.as_retriever()` the same way for either.
- **Two SDKs working together:** the raw `pinecone` SDK handles *index administration* (create/describe), and LangChain's `PineconeVectorStore` handles the *data plane* (embed + upsert + query).
- **Embedding is done inside the store:** `from_documents` / `add_documents` call `embed_documents` internally, so the project never handles raw vectors itself.

7.5 `modules/retriever.py` — Retrieval Strategy

`get_retriever(vectorstore: VectorStore, config: dict) → BaseRetriever` · line 9

`retriever_type`	Construction	Characteristics
`"base"` (default)	LC `vectorstore.as_retriever(search_kwargs={"k": 3})`	Similarity search, top-3 chunks. 1 embedding call per question.
`"multiquery"`	LC `MultiQueryRetriever.from_llm(retriever=…, llm=…)`, LLM = `ChatOpenAI("gpt-4o-mini", T=0)` or `ChatGoogleGenerativeAI("gemini-2.5-flash")`	The LLM writes alternative phrasings of the question; the union of their results is returned. Better recall, higher latency and cost.
other	—	`ValueError("Unsupported retriever_type")`

The LLM provider for multi-query follows `config["llm_provider"]`. The retriever type is read from the **top-level** key `retriever_type` (see finding F-03).

🦜 🔗 LangChain in action

`MultiQueryRetriever.from_llm` builds an internal `prompt | llm | line-parser` LCEL chain that produces the alternative questions, runs the base retriever for each, and returns the unique documents. The returned object is a `BaseRetriever`, so it fits into the RAG chain exactly where the base retriever would.

7.6 `modules/rag_chain.py` — The Hybrid RAG Chain

`build_rag_chain(retriever: BaseRetriever, config: dict) → Runnable[dict, AIMessage]` · line 14

Config key	Default	Effect
`llm_provider`	`"openai"`	Chooses `ChatOpenAI` or `ChatGoogleGenerativeAI`; any other value → `ValueError`.
`llm_temperature`	`0.3`	Sampling temperature for the answer model.
`openai_model`	`"gpt-4o-mini"`	OpenAI chat model name.
`gemini_model`	`"gemini-2.5-flash"`	Gemini chat model name.

INTERNAL ELEMENTS

Element	Description
`system_prompt`	Role instructions telling the model it has (1) a conversation summary and (2) the most recent turns, and should use both.
`prompt`	LC `ChatPromptTemplate` with sections: System Prompt · Conversation Summary Memory · Recent Chat History · Retrieved Context · User Question, followed by guidance to rely on memory for meta-questions ("summarise our chat") and on context + memory otherwise.
`combine_docs(docs)`	Accepts a list (or a dict with `context`); returns the chunk texts joined by blank lines, or *"No relevant context found."* when nothing was retrieved — so the model is told explicitly when there is no context.
`rag_chain`	`RunnableParallel(5 branches) | prompt | llm` — see section 6.3.

🦜🔗 **LangChain in action — the centre of the project**

This module is where LCEL is used most. A single declarative expression combines **retrieval** (`retriever` Runnable), **memory** (config-aware `RunnableLambda`s), **prompting** (`ChatPromptTemplate`) and **generation** (`BaseChatModel`). The result supports `invoke` / `batch` / `stream` and the async versions, and it is traced in LangSmith with no extra code.

7.7 `modules/memory.py` — Hybrid Session Memory

Purpose. Gives each chat session its own memory, made of three per-session stores:

Module-level store	Value type	Purpose
`SESSION_MESSAGE_HISTORY`	LC `InMemoryChatMessageHistory`	Complete list of messages for the session.
`SESSION_SUMMARY_MEMORY`	LC `ConversationSummaryMemory`	LLM-maintained running summary (gpt-4o-mini, T=0), sharing the message history above.
`SESSION_RECENT_MEMORY`	`list[{"user","ai"}]`	Rolling window of the last *n* (default 2) Q&A pairs.

Function	Behaviour
`get_session_message_history(session_id)` l.21	Creates the session's `InMemoryChatMessageHistory` on first use, then returns it.
`get_session_summary_memory(session_id)` l.27	Creates the session's `ConversationSummaryMemory(llm=ChatOpenAI("gpt-4o-mini",0), chat_memory=history, return_messages=True)` on first use, then returns it.
`get_recent_memory(session_id, n=2)` l.38	Returns the last *n* stored turns (or fewer).
`update_recent_memory(session_id, user, ai, n=2)` l.44	Appends `{"user","ai"}` and trims the list to the last *n* entries.
`get_recent_memory_formatted(session_id, n=2)` l.52	Formats the window as `User: …\nAssistant: …` blocks, or *"No recent chat history available."*
`add_memory_to_chain(rag_chain, session_id, enabled=True, recent_n=2)` l.64	If `enabled` is false, returns the chain unchanged (stateless). Otherwise wraps it — see below.
`get_memory_summary(session_id)` l.106	Returns the current summary text (read by the `memory_summary` branch).
`get_recent_context(session_id, n=2)` l.111	Returns the formatted recent window (read by the `recent_memory` branch).

HOW `ADD_MEMORY_TO_CHAIN` WRAPS THE CHAIN

```
def chain_with_summary(input_data, config=None):
    response  = rag_chain.invoke(input_data, config=config)      # 1. run RAG
    summary_memory.save_context({"input": q}, {"output": a})     # 2. update summary (LLM call)
    update_recent_memory(session_id, q, a, n=recent_n)           # 3. update rolling window
    new_summary = summary_memory.predict_new_summary(msgs, old) # 4. re-summarise (2nd LLM call)
    summary_memory._buffer = new_summary                          #    see finding F-05
    return response

return RunnableWithMessageHistory(
    RunnableLambda(chain_with_summary),
    get_session_history=lambda _: get_session_message_history(session_id),
    input_messages_key="question", history_messages_key="history", output_messages_key=None)
```

🦜🔗 LangChain in action

- `RunnableWithMessageHistory` takes the human message from the `question` key, adds stored history under `history`, and records the returned `AIMessage` (with `output_messages_key=None` the whole output is treated as the message).
- `ConversationSummaryMemory` is a classic (pre-LCEL) memory class used here only as a *summary engine*: its `save_context` and `predict_new_summary` run an internal summarisation prompt through the LLM.
- **Hybrid design:** the summary provides long-range, compressed context; the rolling window keeps the exact wording of the latest turns — both are placed in the prompt on every call.

7.8 `app/streamlit_app.py` — UI & Orchestration

Section	Lines	Description
Path bootstrap	5–7	Adds the project root to `sys.path` so `modules.*` imports work when Streamlit is started from any folder.
Config load	21–23	Reads `config/config.json` into the global `CONFIG`.
Session model	27–57	`st.session_state.chat_sessions: {uuid: {name, llm_provider, memory_enabled, chat_history}}` and `active_session`. One session is created automatically; ➕ **New Chat** adds more.
Session selector	61–79	A selectbox of session names maps back to the session UUID.
Per-session settings	85–97	LLM provider (`openai` / `gemini`) and memory toggle, with widget keys unique to each session.
Uploader & ingestion	102–124	Saves the file, then `load_document → split_documents(400, 50) → build_or_update_vectorstore`.
Backend build	129–144	`build_or_update_vectorstore([], CONFIG)` → `get_retriever` → sets `CONFIG["llm_provider"]` from the session → `build_rag_chain` → `add_memory_to_chain`. Errors are shown with `st.error` and execution stops (`st.stop()`).
Chat loop	150–182	Shows earlier messages; for new input, invokes the chain with `configurable.session_id`, renders `response.content`, and catches exceptions so they are shown in the chat.

> 🦜🔗 **LangChain in action**
>
> The UI never works with LangChain objects directly beyond a single `.invoke()` call. The Streamlit session UUID becomes LangChain's `configurable.session_id`, which connects UI sessions to LangChain memory.

7.9 `template.py` — Project Scaffolding

```
create_project_structure() → None
```

Uses a `dict[folder → items]` structure to create the folders (`config, data, vectorstores, modules, app, notebooks`), sub-folders (entries ending in `/`) and empty files (`pathlib.Path.touch`). The `""` key stands for root-level files. Safe to run again: `exist_ok=True`, and `touch` does not truncate existing files.

7.10 `notebooks/RAG_Debugging_Notebook.ipynb` — Test Harness

Cell	Stage tested	What it verifies
0	Setup	Adds the parent folder to `sys.path`; imports loader and splitter; suppresses warnings.

Cell	Stage tested	What it verifies
1	Loader	Loads `sample_text.txt` (creates a placeholder if missing); prints document count and a preview.
2	Splitter	Splits with 200 / 20; prints chunk count, first chunk and its metadata.
3–4	Embeddings	(Commented out) smoke test of `get_embedding_model` for both providers.
5–6	Vector store	Loads config, splits with 200 / 40, builds the FAISS index → creates `notebooks/vectorstores/openai_faiss_index/`.
7	Similarity search	`vectorstore.similarity_search("Explain scikit-learn in short?", k=2)`.
8	Retriever	`get_retriever(...).invoke(query)` — exercises the Runnable retriever interface.
9	Chain + memory	Two-turn conversation for session `test_user_001`; the second question ("summarise our conversation in 3 bullet points") tests that memory works.

8 Configuration Reference

8.1 `config/config.json`

Key	Value	Read by	Notes
`llm_provider`	`"openai"`	rag_chain, retriever	**Active** Overridden at runtime by each session's sidebar choice.
`embedding_provider`	`"openai"`	embedder → utils	**Active** `openai` | `gemini`. Changing it creates a separate index (the provider is part of the path/name).
`vector_store.type`	`"faiss"`	embedder	**Active** `faiss` | `pinecone`.
`vector_store.local.faiss_path`	`"vectorstores/faiss_index"`	—	**Unused** Path is derived from the cwd instead.
`vector_store.cloud.index_name`	`"rag-assistant-index"`	embedder	**Active** `-<provider>` is appended.
`vector_store.cloud.namespace`	`"default"`	embedder	**Active** Pinecone namespace.
`vector_store.cloud.environment`	`"gcp-starter"`	embedder	**Fallback** Used only when `PINECONE_ENV` is not set; `*-starter` selects `PodSpec`.
`vector_store.cloud.provider` / `dimension`	`"faiss"` / `1536`	—	**Unused** Dimension is detected at runtime.
`retriever.type` / `top_k`	`"base"` / `3`	—	**Unused** Code reads top-level `retriever_type`; k is hard-coded to 3.
`prompt.system_prompt` / `temperature` / `max_tokens`	…	—	**Unused** Prompt is hard-coded; temperature read from `llm_temperature`.
`memory.enabled_default` / `max_memory_window`	`true` / `5`	—	**Unused** UI defaults to on; window is `recent_n=2`.
`chunking.chunk_size` / `chunk_overlap`	`500` / `100`	—	**Unused** App passes 400 / 50.
(absent) `llm_temperature`, `openai_model`, `gemini_model`, `retriever_type`	—	rag_chain, retriever	**Optional** Can be added at top level to override the code defaults.

8.2 Environment Variables (`config/.env`)

Variable	Required when	Purpose
`OPENAI_API_KEY`	Always (in practice)	OpenAI embeddings/chat. **Also required for summary memory even when Gemini is selected**, because `memory.py` always uses `ChatOpenAI`.
`GOOGLE_API_KEY`	Gemini selected	Gemini chat and/or embeddings.
`PINECONE_API_KEY`	`vector_store.type = pinecone`	Pinecone authentication.
`PINECONE_ENV`	Optional	Region (e.g. `us-east-1`); takes precedence over `config.json` .
`LANGCHAIN_TRACING_V2` , `LANGCHAIN_API_KEY` , `LANGCHAIN_PROJECT`	Optional	Turns on LangSmith tracing of every chain run (the `langsmith` package is already installed).

9 Installation & Operation

9.1 Prerequisites

Python	3.10 – 3.12 (developed on 3.12)
Accounts / keys	OpenAI API key (required); Google AI Studio key (for Gemini); Pinecone key (for cloud store)
OS	Windows, macOS or Linux

9.2 Setup

```
# 1. Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate            # Windows   (macOS/Linux: source .venv/bin/activate)

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure secrets — copy the template into config/.env and fill in keys
copy env-sample.txt config\.env   # Windows   (macOS/Linux: cp env-sample.txt config/.env)
```

9.3 Running

```
# From the project root (important: FAISS index path is relative to the working directory)
streamlit run app/streamlit_app.py
```

1. On first launch no index exists, so the page shows *"No documents provided and FAISS index not found."* — upload a document in the sidebar to create it.
2. Ask questions in the chat box. Use ➕ **New Chat** for separate conversations, and the sidebar to switch LLM provider or turn memory on/off for each chat.
3. For step-by-step debugging, open `notebooks/RAG_Debugging_Notebook.ipynb` and run the cells in order.

9.4 Switching Providers

Goal	Change
Answer with Gemini	Sidebar → *LLM Provider* → `gemini` (per session, no restart).
Embed with Gemini	`"embedding_provider": "gemini"` → re-upload documents (a new index is created).
Use Pinecone	`"vector_store": {"type": "pinecone", …}` + `PINECONE_API_KEY` (+ `PINECONE_ENV`).
Multi-query retrieval	Add top-level `"retriever_type": "multiquery"`.
Change model / temperature	Add top-level `"openai_model"`, `"gemini_model"`, `"llm_temperature"`.

10 Engineering Review

A code review carried out while preparing this document found the following issues. They are listed to help with future work; none of them stops the main flow (upload → ask → answer) from working.

10.1 Findings Register

ID	Severity	Location	Finding	Recommendation
F-01	High	`streamlit_app.py:111`	Streamlit reruns the script on every interaction and the uploader keeps its file, so the `if uploaded_file:` block runs again on every chat message. The same document is re-embedded and **added to FAISS again each time** (duplicate chunks, extra embedding cost).	Record processed file names/hashes in `st.session_state` and skip files already ingested.
F-02	High	`memory.py:30`	Summary memory always uses `ChatOpenAI`, so Gemini-only setups fail without an OpenAI key, and summarisation calls go to OpenAI even in Gemini sessions.	Pass the session's chat model into `get_session_summary_memory`.
F-03	Medium	`retriever.py:11`	Reads `config["retriever_type"]`, but `config.json` defines `retriever.type` / `top_k`, which are therefore ignored.	Read `config["retriever"]["type"]` and `top_k`.
F-04	Medium	`retriever.py:28`	Typo `search_kwards` → the multi-query base retriever ignores k=3 and falls back to the library default (k=4).	Change it to `search_kwargs`.
F-05	Medium	`memory.py:84–87`	`save_context` already updates the summary. The second `predict_new_summary` call is an extra LLM call, and its result is stored in `_buffer`, a field the class never reads (the real field is `buffer`), so it is thrown away.	Remove lines 84–87, or assign to `summary_memory.buffer` and drop the `save_context` summary.
F-06	Medium	`memory.py:78` / `:96`	`save_context` and `RunnableWithMessageHistory` both write to the same `InMemoryChatMessageHistory`, so each turn is stored twice.	Give the summary memory its own history object, or let only one component write.
F-07	Medium	`embedder.py:55`	The FAISS path depends on the current working directory. The notebook and the app therefore use different indexes (`notebooks/vectorstores` vs `./vectorstores`), and `vector_store.local.faiss_path` is ignored.	Resolve the path from the project root and use `faiss_path` from config.
F-08	Medium	`streamlit_app.py:129–144`	The vector store, retriever and chain are rebuilt (and FAISS is saved to disk) on every rerun.	Cache them with `@st.cache_resource`, keyed on provider/config.

ID	Severity	Location	Finding	Recommendation
F-09	Low	`config.json`	The `prompt`, `memory`, `chunking` and `retriever` sections are not read; values are hard-coded instead.	Read these sections in the code so `config.json` is the single source of truth.
F-10	Low	`loader.py:53`	`"md"` is missing its leading dot; `.doc` / `.docs` cannot be read by python-docx.	Use `".md"`; remove `.doc` / `.docs` from the list.
F-11	Low	retriever, splitter, embedder	Unused imports (`LLMChain`, `PromptTemplate`, `get_embedding_model`, `Chroma`) and a duplicate splitter import.	Remove them.
F-12	Low	memory, rag_chain, retriever	Old import paths (`langchain.schema.runnable`, `langchain.prompts`); `ConversationSummaryMemory` is deprecated in LangChain 0.3.	Import from `langchain_core.*`; build the summary step as a small LCEL chain.
F-13	Low	`streamlit_app.py:108`	The `os.path.join(...)` result is never used.	Delete the line.
F-14	Low	`gradio_app.py`, `run_app.bat`	Empty placeholders.	Implement them or remove them from the repository.

10.2 Recommended Roadmap

PHASE 1 · CORRECTNESS

- Fix F-01 (re-ingestion)
- Fix F-03/F-04 (retriever config)
- Fix F-05/F-06 (memory)
- Use the session's LLM for summaries (F-02)

PHASE 2 · LANGCHAIN MODERNISATION

- Move imports to `langchain_core`
- Replace `ConversationSummaryMemory` with an LCEL summariser
- Show answers token by token with `chain.stream()` + `st.write_stream`
- Return source citations from chunk metadata

PHASE 3 · PRODUCTION

- Cache resources (F-08)
- Persistent chat history (Redis / SQL)
- Turn on LangSmith tracing and evaluation
- Gradio UI + launcher script

11 Glossary

RAG	Retrieval-Augmented Generation — retrieving relevant passages from your own data and adding them to the LLM prompt so the answer is based on that data.
LangChain	Open-source framework that provides standard interfaces and integrations for building LLM applications.
LCEL	LangChain Expression Language — the `|` -based way of combining Runnables into chains that support invoke/batch/stream and async.
Runnable	LangChain's base interface for any step in a chain (prompts, models, retrievers, lambdas, parallel maps).
Document	LangChain's text container: `page_content` + `metadata`.
Chunk	A piece of a document produced by a text splitter; the unit that is embedded and retrieved.
Embedding	A list of numbers (vector) that represents the meaning of a text; similar texts produce nearby vectors.
Vector store	A database that stores embeddings and finds the nearest ones to a query vector (FAISS, Pinecone).
Retriever	A Runnable that maps a query string to a list of relevant Documents.
Multi-Query Retrieval	Using an LLM to rewrite a question in several ways, then merging the results of each search to improve recall.
Summary memory	An LLM-maintained short summary of a conversation, used to keep long-range context.
Rolling window memory	The last *n* turns kept word for word, used for short-term context.
RunnableConfig / configurable	Runtime settings passed alongside a chain's input (e.g. `session_id`, callbacks, tags) and passed down to every nested step.
Namespace (Pinecone)	A logical partition inside a Pinecone index, used to separate data sets.
LangSmith	LangChain's tracing and evaluation platform for inspecting chain runs.

Capstone RAG Chatbot — Technical Documentation

Revision 1.0 · October 2026 · Prepared for the IIT Patna Module 5 Capstone. Generated from the source code at the time of writing; line references apply to that version.